

A Large-scale Customer-Facility Network Model for Customer Service Centre Location Applications

David Chalupa¹ and Peter Nielsen²

Operations Research Group
Department of Materials and Production
Aalborg University
Fibigerstræde 16, Aalborg 9220, Denmark
`{dc,peter}@m-tech.aau.dk`

Abstract. We propose a large-scale sparse customer-facility network model that allows a customer to be assigned only to facilities within the vicinity of a customer. In this model, customer-facility distances are integer values representing zones. Experimental results are presented for large instances with up to 100,000 customers and 100 potential facility sites. A mixed-integer linear programming solver reveals large gaps in suboptimal solutions and lower bounds provided, even with a considerable computational effort. Two simple but scalable local search heuristics are computationally investigated, revealing their potential for solving such large-scale problems in practice.

Keywords: facility location problem; customer-facility networks; sparse problems; customer service centre location

1 Introduction

Facility location problems are among the fundamental problems in operations research and manufacturing [1], with wide applications such as supply chain [2] or manufacturing network design [3]. Many of these applications require solving of large-scale instances of the problem, which is a computationally hard task, given the NP-hardness of the problem [4]. Likewise the problem has many similarities to a number of well-known assignment problems found in literature such job-shop scheduling [5], resource allocation problems [6], knapsack problems [7], project selection problems [8] and container placement problems [9].

For applications such as customer service centre location, the problem can get very large in terms of the numbers of customers to assign. This type of scale is also often found in transportation and knapsack problems. Facility location problems have been tackled for moderately sized but structurally challenging instances by a number of stochastic algorithms [10]. These include including local search [11], tabu search [12,13,14], genetic algorithms [15], evolutionary simulated annealing [16] or particle swarm optimisation [17]. Related problems

have been tackled for relatively large numbers of customers, a recent study solved the competitive facility location problem efficiently for instances with around 7,000 customers [18].

Yet, to the best of our knowledge, algorithmic requirements for instances with tens or even hundreds of thousands of customers remain largely unexplored. This is interesting as the nature of the problem lends the results to have implication for other related problems. In this paper, we develop a sparse customer-facility network model, in which the distances represent zones and a customer can be assigned to a facility only within a limited zone. This has a simple real-world interpretation: it makes sense to restrict the set of potential facility sites to serve the customer only to the ones within their vicinity. This keeps the problem instance sparse and is likely to improve on scalability of problem solving techniques for the problem in practice. It also opens the way to relating the problem closely to other location problems in complex networked systems, such as the k -reachability problems [19].

Contributions. Our sparse customer-facility network model works with randomly generated facility costs and integer distances that represent zones. A zone threshold is introduced and an edge between a customer and potential a facility site is established only if the zone is within the threshold. This leads to a large but sparse networks, for which specific computational techniques are available [20].

Experimental results are presented for large-scale implementations of the model with 10,000 customers, 100,000 customers and 100 potential facility sites. The facility costs are generated uniformly at random and the customers are uniformly assigned to 10 zones for each facility, limiting the connections only to zones 1 and 2. It is worth noting that we treat zones as symmetric distances, i.e. assigning a customer to a zone for a facility, and assigning a facility to a zone for a customer essentially have the same meaning.

We solve the problem using scalable local search (LS) and randomised local search (RLS) algorithms that we propose for problems of this scale. Performance of these local search algorithms in solving various problems is widely studied in evolutionary computation theory [21]. Note that these algorithms were chosen, since simple algorithms usually have good scaling properties. In addition, just sampling a promising local optimum is a costly computational operation at such a scale.

Compared to a mixed-integer linear programming solver, LS and RLS provided solutions of very good quality much faster, and were the only techniques producing solutions to the instances with 100,000 customers. Interestingly, the gap between the average and mean performances of LS and RLS seems to narrow down with growing instance size. However, while LS seems to be a technique suitable for finding a good local optimum quickly, RLS exhibits high variance in its results. Using these instances and algorithms allowed us to explore the implications of solving large-scale but sparse instances of general assignment problems on search landscape and algorithm design. As the problem studied is

chosen as an example of such an assignment problem, these implications reach far beyond the facility location problem.

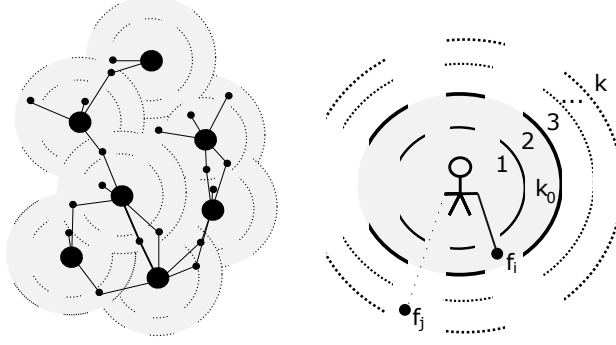


Fig. 1. Illustration of a simple customer-facility network according to our model (on the left-hand side) and distances represented by k zones (on the right-hand side). Facilities are depicted as the larger circles. A customer can be assigned only to a facility within the corresponding zones. Note that the spatial depiction is only used for illustration. Each customer can be assigned to a facility if it lays within zone up to the threshold k_0 , leading a sparse structure of the problem. In our experiments, we will further assign the zones at random.

2 Customer-Facility Network Model for Large-scale Applications

In order to represent the facility location problem as a sparse problem, one needs to slightly adapt the traditional integer linear programming (ILP) formulation of the problem. The following is an adaptation of the most widely used formulation of the problem [12]:

$$\min \sum_{i=1}^n f_i y_i + \sum_{\{v_i^f, v_j^c\} \in E} c_{ij} x_{ij}, \quad (1)$$

s.t.

$$\sum_{\{v_i^f, v_j^c\} \in E} x_{ij} = 1, \quad j = 1, \dots, m, \quad (2)$$

$$x_{ij} \leq y_i, \quad i = 1, \dots, n, \quad \{v_i^f, v_j^c\} \in E, \quad (3)$$

where f_i is the cost of facility i and c_{ij} is the integer zone value for customer j if assigned to facility i . The difference between this and the classical formulation

is that one now operates within a network of customers and facilities that can be formalised as a graph $G = [V, E]$. $V = [\mathcal{C}, \mathcal{F}]$ represents the vertices, dividing customers and potential facility sites into separate sets, and E is a set of connections of customers to potential facility sites. Note that $\{v_i^f, v_j^c\} \in E$ if zone c_{ij} is within a bounded interval for customer j and facility site i . This creates a sparse bipartite networked structure, as illustrated in Figure 1 (on the left-hand side). The technical details of this restriction will be discussed further in the next paragraphs. Constraint (2) traditionally requires that a customer is assigned to a single facility. Here we restrict this so that the customer is assigned locally, i.e. the set of feasible solutions is restricted only to assignments within E .

Figure 1 (on the right-hand side) illustrates the principles of zones within our model. For a customer j and facility i , the facility is situated within zone c_{ij} . There are k zones overall, i.e. $1 \leq c_{ij} \leq k$. Then, we say that $\{v_i^f, v_j^c\} \in E$ if $c_{ij} \leq k_0$, where k_0 is the zone threshold. This threshold limits the possible assignment of customers only to the closest facilities and ensures that the problem instance becomes sparse.

In summary, our model represents large-scale sparse instances of the facility location problem with the following properties:

- n potential facility sites and m customers, with m potentially being a very high number;
- uniform distribution of facility costs $f_i \in \{1, 2, \dots, f_{max}\}$ for each facility i ;
- uniform distribution of zone assignments $c_{ij} \in \{1, 2, \dots, k\}$ for each customer j and facility i ;
- zone threshold $1 \leq k_0 \leq k$ to regulate local properties and sparseness of the instance.

This slight modification of the formulation of the facility location problem allowed us to solve it using LS and RLS. These algorithms are introduced below. LS and RLS are expected to be able to deliver reasonable results even for very large-scale problem instances. Importantly, we assume that the attributes exhibited in this problem are somewhat representative of also other types of assignment problems. This highlights the relevance of our study for practice.

In addition, we are interested in how the performance of LS and RLS scales from the smaller instances to the very large instances. We are especially interested in whether the performance scales proportionally. In the following, we discover that this is not true, but rather we see that LS and RLS behave somewhat differently. This uncovers the opportunities not only to exploit these algorithms in facility location problems, but also leading to results reaching beyond this problem.

3 Scalable Local Search Algorithms

To solve the problem efficiently for the instance sizes discussed, we will use two simple but efficient and scalable local search algorithms [22,21]. Both of

the algorithms operate in the search space of binary strings $y \in \{0, 1\}^n$, where element y_i represents a decision variable determining whether facility i is open or closed. The values of x_{ij} are computed by assigning each customer to the closest facility. In our design, both algorithms will start with binary string $1, 1, \dots, 1$, i.e. all facilities will be open at the start. The reasons for this decision are two-fold. Firstly, both algorithms will always start at the same point in the search space, which eliminates one source of uncertainty in our computational investigation. Secondly, this is a solution that will likely be feasible also in further constrained variants of the problem, since we aim also for an easy generalisation to related problems. The stopping criteria for both algorithms will depend on the number of potential facility sites, following the increasingly popular fixed budget computation paradigm [23].

Local search (LS). Let $y \in \{0, 1\}^n$ be the current solution. The neighbourhood of y consists of all binary strings obtained by flipping a single bit, i.e. $\mathcal{N}(y) = \{y' \mid y' = y + c, \sum_{i=1}^n c_i = 1\}$. Let the objective function in (2) be denoted by $J(y)$. Then, LS picks $y' = \arg \min_{y' \in \mathcal{N}(y)} J(y)$, i.e. it chooses the move that minimises the objective within the neighbourhood. In our further experiments, LS will be stopped after n iterations, where n is the number of facilities.

Randomised local search (RLS). RLS works with the same neighbourhood structure $\mathcal{N}(y)$ as LS. However, in each iteration, it always chooses a facility site i and attempts to flip bit y_i , effectively opening or closing the corresponding facility. This leads to a new candidate solution y' . This solution is accepted simply if $J(y') \leq J(y)$, i.e. the new solution is at least as good as the current solution. In our further experiments, RLS will be stopped after n^2 iterations. This procedure effectively evaluates the same number of candidate solutions as LS, since LS scans all n solution in the neighbourhood in each iteration, while RLS attempts to evaluate only a single one.

To implement the checking of the potential moves efficiently, one can use a collection of efficient data structures, similarly to the design of tabu search algorithms [14]. For each facility, we explicitly store the list of customers assigned to it. Conversely, we also store the assignment vector determining the corresponding facility for each customer. This way, the objective value can first be recalculated, before a move is accepted. These structures are then updated after accepting the move.

4 Experimental Results

In order to evaluate our model, we have performed experiments for two sets of large-scale problem instances generated for 10,000 and 100,000 customers. We generated 10 problem instances of size 10000×100 and 100000×100 , with a maximum facility cost $f_{max} = 100$ and a maximum of $k = 10$ zones. For a connection between a customer and a potential facility site to be established, each customer has to be within zone $k_0 = 2$ (or lower).

The code of LS and RLS was written in C++ using Qt and compiled using the 32-bit MinGW compiler under Windows 10. We also used the mixed-integer lin-

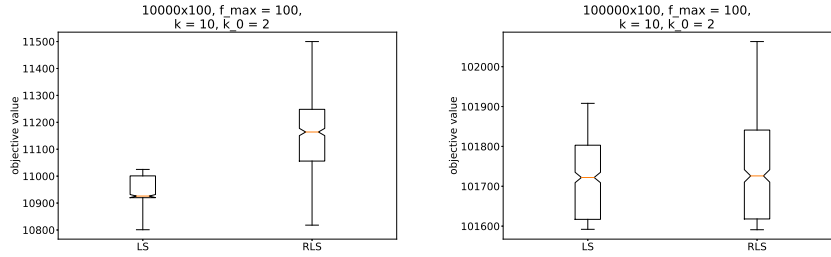


Fig. 2. Box-whisker plots depicting the results obtained by LS and RLS for the instances of size 10000×100 (on the left-hand side) and 100000×100 (on the right-hand side).

ear programming branch-and-cut solving utility CBC from the COIN-OR package [24,25]. This was to see how difficult it is to solve the problem into optimality at this scale and obtain worst-case bounds on the quality of the solutions obtained by LS and RLS. All experiments were performed on a machine with Intel Core i7-6820 CPU @ 2.70 GHz and 32 GB RAM. We used pre-compiled binaries of the COIN-OR package compiled by the 64-bit Intel 11.1 compiler.

In Figure 3, we present the box-whisker plots obtained for LS and RLS for the corresponding 10000×100 (on the left-hand side) and 100000×100 (on the right-hand side) instances. These are obtained for 10 independently generated instances and 100 runs of both LS and RLS for each of the instances. The results show that LS outperforms RLS in terms of solution quality for the smaller instances. However, for larger instances, the performances of LS and RLS are much more even. Note that the difference between these two instance types is only in the number of customers, all other parameters were the same. In a more detailed analysis, we will see that the performance of the algorithms is more intricate than it seems on the surface. For some of the instances, RLS actually performed better than LS in the best runs, even though it exhibited weaker average and median performances.

Table 1 provides detailed results for the 10000×100 instances. These show that both LS and RLS took between 12 and 15 seconds to solve the problem. One can see that the performance of LS is much more stable, while the variance of results obtained by RLS is much higher. This is likely related to the random sampling of the moves performed by RLS, in contrast to the steepest descent strategy of LS. LS emerged as the better of the two strategies for these instances, even though it is worth mentioning that RLS sampled a better solution for instance 9.

Regarding the performance of CBC in solving these instances exactly, one can see that for all apart of one instance, LS or RLS found a better solution than CBC. All the above solutions found by the CBC are found only by terminating the search after 24 hours, still leaving a wide gap between the solution found

and the lower bound. This shows that heuristics are indeed very useful especially for such large-scale sparse instances of the problem. It is also worth noting the relatively distant optimal solutions for the LP relaxations and a very limited progress of CBC in improving the lower bound. Given the relatively generous time limit given to CBC, it seems that these instances are not only large but also quite intricate in terms of the search landscape.

This finding complements a large body of work that focused on moderately large but structurally highly intricate instances [12,10,17,15,13,14,16]. It also identifies algorithms for these instance scales as a largely unexplored but practically very relevant area.

In Table 2, we provide the detailed results obtained for the 100000×100 instances. For this instance scale, we already observed very high computational demands of CBC. While the LP relaxation of problem could be solved within hours, already the preparation phase for the branch-and-cut procedure took more than 24 hours.

This renders heuristics to be the only practical choice for this instance scale. Both LS and RLS produced solutions in two to four minutes, with LS being slightly quicker, which is likely related to less randomness within the process of LS and less frequent solution updates. One can also see that the performance of LS is quite stable in different runs, often leading to matching average and best upper bounds. However, we also observed that it at times converged to solutions that can be easily identified as suboptimal. This is observed particularly for instances 1, 4 and 10, for which RLS found better solutions than LS in the best runs. However, the inverse was observed for instance 9. This shows that while LS is better both in its average and median performances, the variance in the behaviour of RLS can have the advantage of avoiding local optima or escaping them.

It is also worth noting that the performance gap between the average and median performances of LS and RLS seems to shrink with growing number of customers. While the gap between the performances is quite wide for 10,000 customers, the average and median performances seem much closer for 100,000 customers. We have tested this further for 200,000 customers and the gap seems to close further. However, while LS seems to behave quite predictably, often converging to a limited number of local optima, RLS preserves a much higher variance in its behaviour. This indicates that LS and RLS have complementary qualities for very large-scale instances. While LS is a good strategy to sample promising local optima of these instances quickly, RLS can at times sample solutions of better quality and be used as a multi-start algorithm or as a procedure for escaping local optima.

These results sketch quite a complex picture of the performance of different strategies in our model. The problem becomes increasingly difficult for the branch-and-cut exact solver not only with the number of facilities but also with the number of customers in the model. For 100,000 customers, the exact solver with a 24 hour time limit was only able to compute the solution to the LP relaxation of the problem. Combined with the fact that these lower bounds are

distant, this highlights the demand for efficient scalable heuristics for solving the problem at such a scale. The performance of heuristics themselves is also quite complex. LS outperformed RLS for the model with 10,000 customers but with a growing number of them, the gap between the performances of LS and RLS is gradually closing. While the gap between average objective values obtained by LS and RLS was 228.1 in favour of LS for 10,000 customers, it becomes only 18.3 for 100,000 customers. This can also be seen in the gap between the best objective values that was 40.7 in favour of LS for 10,000 customers. However, this gap became 0.6 in favour of RLS for 100,000 customers. In contrast to this, the gap between the best values obtained by the better of the two heuristics and the lower bound seems to widen with growing number of customers. This gap was 132.1 for 10,000 customers and 411.6 for 100,000 customers. LS is therefore a good choice for finding a good solution rapidly, while RLS preserves variability in its performance in large-scale, possibly leading to improved results. This suggests that a multi-start or a population-based local search heuristic may be a good direction for future algorithm design for this scale.

5 Conclusions

In this paper, we proposed a new large-scale sparse customer-facility network model for the facility location problem. This model is especially relevant for applications with high numbers of customers, such as customer service centre location.

The model works with facility costs that are distributed uniformly between 1 and a value f_{max} , and the distances were represented by zones, assigning each pair customer-facility with a zone number between 1 and k . We then introduce a distance threshold k_0 and allow a customer to be assigned only to a facility within zone at most k_0 . This leads to large but sparse problem instances, allowing to model scenarios with potentially very high numbers of customers, with potential assignments limited only to the close vicinity of a customer or a facility.

In our experiments, we explored instances with 10,000 customers, 100,000 customers, 100 potential facility sites, and 10 zones, with connections between customers and facility possible only within zones 1 and 2. To the best of our knowledge, the problem has not yet been widely tackled in such a large scale.

Our experimental results were presented for a branch-and-cut mixed-integer linear programming solver, as well as two scalable local search heuristics: local search (LS) and randomised local search (RLS). These have uncovered a complex pattern of efficiency of different techniques to solve the problem in such a scale. The branch-and-cut solver found a relatively wide gap between the solution found and a lower bound for the model with 10,000 customers. For 100,000 customers, it was only able to produce a lower bound within 24 hours.

Regarding the heuristics, while LS provided a solution of good quality rapidly, RLS exhibits higher variance in its performance. However, RLS is at times able to sample a better solution than LS.

Table 1. A detailed breakdown of the results obtained for instances with 10,000 customers and 100 potential facility sites. The results obtained by solving the LP relaxation, branch-and-cut (both using CBC), LS and RLS are provided. CPU time measures are provided to quantify the scalability.

instance	LP relaxation				branch-and-cut		LS		RLS			
	lower bound	CPU time	upper bound	time	lower bound	time	upper bound	average	best upper	time	best upper	CPU time
1	10787	196 s	10928	10788	24 h	10920	10920	13 s	11156	10955	13 s	13 s
2	10811	203 s	10971	10812	24 h	10926	10926	12 s	11154	10977	12 s	12 s
3	10674	168 s	10796	10675	24 h	10801	10801	12 s	10968	10818	13 s	13 s
4	10852	175 s	11007	10855	24 h	11001	11001	12 s	11272	11066	12 s	12 s
5	10848	159 s	11069	10849	24 h	11012	11007	12 s	11225	11012	12 s	12 s
6	10845	167 s	11072	10846	24 h	11020	11020	12 s	11318	11114	12 s	12 s
7	10682	173 s	10805	10683	24 h	10797	10797	12 s	11038	10875	13 s	13 s
8	10820	119 s	10956	10822	24 h	10958	10956	13 s	11199	11008	13 s	13 s
9	10801	109 s	10961	10802	24 h	10944	10944	13 s	11153	10925	14 s	14 s
10	10713	109 s	10782	10719	24 h	10782	10782	13 s	10965	10811	15 s	15 s

Table 2. A detailed breakdown of the results obtained for instances with 100,000 customers and 100 potential facility sites. The results obtained by solving the LP relaxation, branch-and-cut (both using CBC), LS and RLS are provided. CPU time measures are provided to quantify the scalability.

instance	LP relaxation		branch-and-cut		LS		RLS	
	lower bound	CPU time	upper bound	lower bound	CPU time	average best upper bound	CPU time	average best upper bound
1	101293	22381 s	timeout		101592	101592	140 s	101598
2	101379	23973 s			101803	101801	150 s	101823
3	101345	23421 s			101722	101722	161 s	101727
4	101420	24373 s			101908	101908	163 s	101949
5	101269	24052 s			101617	101617	183 s	101623
6	101180	18749 s			101439	101439	159 s	101451
7	101287	26976 s			101632	101632	155 s	101649
8	101357	27173 s			101837	101837	175 s	101848
9	101524	27467 s			102173	102167	154 s	102187
10	101316	26418 s			101777	101777	162 s	101828
								101774

In addition, the gap between the performances of LS and RLS seems to become more narrow with growing instance size, while the variance in performance of RLS is preserved.

We believe that our findings will lead to future investigations of problem structure, scale and their impact on algorithm design. It seems that there are intricate connections between these properties of a problem instance and the right choice of an algorithm to solve such an instance. The real-world relevance of the scenarios investigated in this paper could also lead to further interesting large-scale practical applications.

References

1. Melo, M.T., Nickel, S., Saldanha-Da-Gama, F.: Facility location and supply chain management—a review. *European journal of operational research* **196**(2) (2009) 401–412
2. Daskin, M.S., Snyder, L.V., Berger, R.T.: Facility location in supply chain design. *Logistics systems: Design and optimization* (2005) 39–65
3. Mourtzis, D., Doukas, M., Vandra, C.: Mobile apps for product customisation and design of manufacturing networks. *Manufacturing Letters* **2**(2) (2014) 30–34
4. Nimrod, M., Arie, T.: On the complexity of locating linear facilities in the plane. *Operations Research Letters* **1**(5) (1982) 194 – 197
5. Applegate, D., Cook, W.: A computational study of the job-shop scheduling problem. *ORSA Journal on computing* **3**(2) (1991) 149–156
6. Lee, Z.J., Lee, C.Y.: A hybrid search algorithm with heuristics for resource allocation problem. *Information Sciences* **173**(1) (2005) 155–167
7. García-Martínez, C., Rodríguez, F.J., Lozano, M.: Tabu-enhanced iterated greedy algorithm: a case study in the quadratic multiple knapsack problem. *European Journal of Operational Research* **232**(3) (2014) 454–463
8. Relich, M., Pawlewski, P.: A multi-agent system for selecting portfolio of new product development projects. *Communications in Computer and Information Science* **524** (2015) 102–114
9. Do, N.A.D., Nielsen, I.E., Chen, G., Nielsen, P.: A simulation-based genetic algorithm approach for reducing emissions from import container pick-up operation at container terminal. *Annals of Operations Research* **242**(2) (2016) 285–301
10. Arostegui, M.A., Kadipasaoglu, S.N., Khumawala, B.M.: An empirical comparison of tabu search, simulated annealing, and genetic algorithms for facilities location problems. *International Journal of Production Economics* **103**(2) (2006) 742–754
11. Arya, V., Garg, N., Khandekar, R., Meyerson, A., Munagala, K., Pandit, V.: Local search heuristics for k-median and facility location problems. *SIAM Journal on computing* **33**(3) (2004) 544–562
12. Al-Sultan, K.S., Al-Fawzan, M.A.: A tabu search approach to the uncapacitated facility location problem. *Annals of Operations Research* **86** (1999) 91–103
13. Michel, L., Van Hentenryck, P.: A simple tabu search for warehouse location. *European Journal of Operational Research* **157**(3) (2004) 576–591
14. Sun, M.: Solving the uncapacitated facility location problem using tabu search. *Computers & Operations Research* **33**(9) (2006) 2563–2589
15. Jaramillo, J.H., Bhadury, J., Batta, R.: On the use of genetic algorithms to solve location problems. *Computers & Operations Research* **29**(6) (2002) 761–779

16. Yigit, V., Aydin, M.E., Turkbey, O.: Solving large-scale uncapacitated facility location problems with evolutionary simulated annealing. *International Journal of Production Research* **44**(22) (2006) 4773–4791
17. Guner, A.R., Sevkli, M.: A discrete particle swarm optimization algorithm for uncapacitated facility location problem. *Journal of Artificial Evolution and Applications* **2008** (2008)
18. Lančinskas, A., Fernández, P., Pelegín, B., Žilinskas, J.: Improving solution of discrete competitive facility location problems. *Optimization Letters* **11**(2) (2017) 259–270
19. Chalupa, D., Blum, C.: Mining k-reachable sets in real-world networks using domination in shortcut graphs. *Journal of Computational Science* **22** (2017) 1 – 14
20. Sun, J., Xie, Y., Zhang, H., Faloutsos, C.: Less is more: Sparse graph mining with compact matrix decomposition. *Statistical Analysis and Data Mining: The ASA Data Science Journal* **1**(1) (2008) 6–22
21. Neumann, F., Witt, C.: *Bioinspired computation in combinatorial optimization: algorithms and their computational complexity*, Springer (2010)
22. Chalupa, D., Hawick, K.A., Walker, J.A.: Hybrid bridge-based memetic algorithms for finding bottlenecks in complex networks. *Big Data Research* (2018) to appear
23. Jansen, T., Zarges, C.: Performance analysis of randomised search heuristics operating with a fixed budget. *Theor. Comput. Sci.* **545** (2014) 39–58
24. Bonami, P., Biegler, L.T., Conn, A.R., Cornuéjols, G., Grossmann, I.E., Laird, C. D.and Lee, J., Lodi, A., Margot, F., Sawaya, N., et al.: An algorithmic framework for convex mixed integer nonlinear programs. *Discrete Optimization* **5**(2) (2008) 186–204
25. Linderoth, J.T., Lodi, A.: *MILP software*. Wiley encyclopedia of operations research and management science (2011)